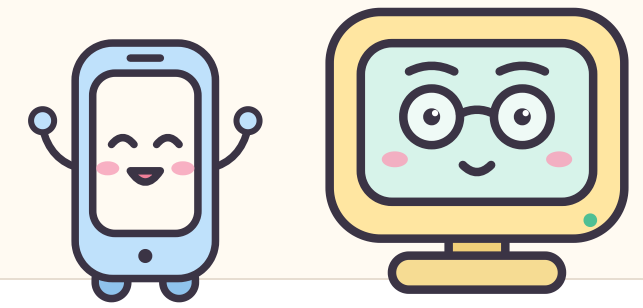




第 12 集 · 约 28 分钟

服务端⑤：配对

每分钟只许猜 20 次，一个月也能猜中

12

从零写出 LanShare

这一集你会学到

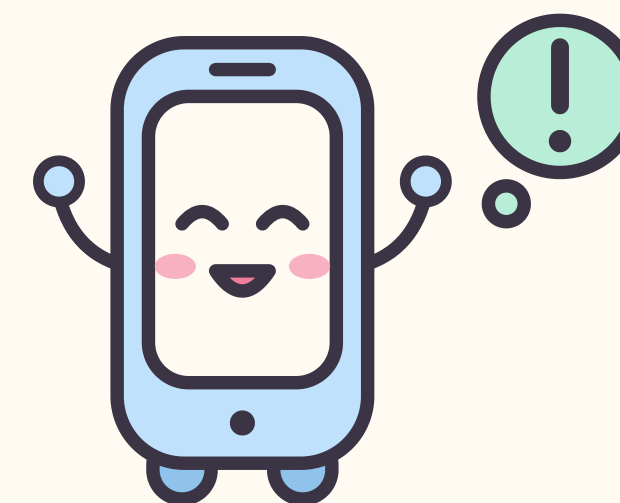
1. 扫码握手 hello：验标签、建会话，重放旧握手也复活不了会话
2. 口令配对的服务端：两步 SPAKE2，做了一半的先放进 pending
3. VecDeque 做 60 秒滑动窗口限速；累计失败就暂停口令登录（审查 I-1）
4. “检查 + 占位”放进同一次加锁（复验 N-4）
5. 握手正文只收 JSON，限量、限时（审查 M-2、第三轮复验 R3-1）



先算一笔账

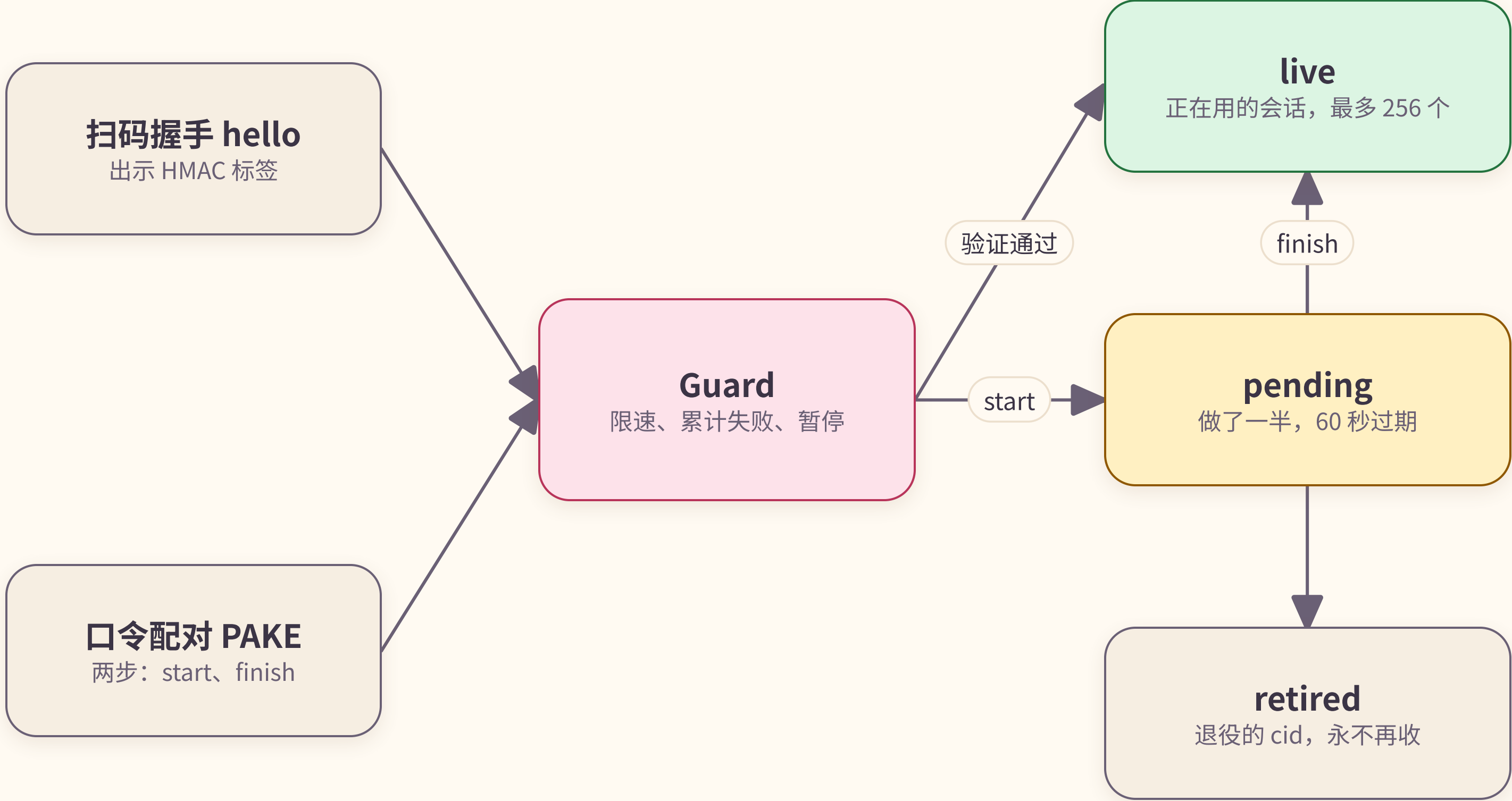
口令是 6 位数字，老电脑每分钟最多受理 20 次尝试。捣蛋鬼要猜多久，才有一半的把握猜中？

大约一个月。每天 28800 次，30 天约 58%。限速只管住了速度，没管住总量。





配对要用到的几张表





auth.rs: 开头



auth.rs crates/lanshare/src/server/

第 4-23 行 • 共 23 行

```
4 use std::net::{IpAddr, SocketAddr};
5 use std::time::{Duration, Instant};
6
7 use axum::Json;
8 use axum::body::Bytes;
9 use axum::extract::{ConnectInfo, Request};
10 use axum::http::{HeaderMap, StatusCode, header};
11 use axum::response::{IntoResponse, Response};
12 use base64::Engine;
13 use lanshare_proto::{self as proto, ChannelKeys, ReplayWindow};
14 use rand_core::OsRng;
15 use serde::Deserialize;
16 use serde::de::DeserializeOwned;
17 use serde_json::json;
18
19 use super::St;
20
21 const WINDOW: Duration = Duration::from_secs(60);
22 const PAKE_TTL: Duration = Duration::from_secs(60);
23 const MAX_PENDING_PAKE: usize = 64;
```



auth.rs: 会话表

auth.rs

crates/lanshare/src/server/

第 18-37 行 • 共 37 行

```
18
19 use super::St;
20
21 const WINDOW: Duration = Duration::from_secs(60);
22 const PAKE_TTL: Duration = Duration::from_secs(60);
23 const MAX_PENDING_PAKE: usize = 64;
24
25 pub(crate) struct Session {
26     pub(crate) keys: ChannelKeys,
27     pub(crate) window: ReplayWindow,
28     last_used: Instant,
29 }
30
31 #[derive(Default)]
32 pub(crate) struct Sessions {
33     live: HashMap<[u8; 16], Session>,
34     /// 被淘汰或配对失败过的 cid: 本次运行内永不再接受, 防止重放旧握手“复活”会话、计数器归零。
35     retired: HashSet<[u8; 16]>,
36     pending: HashMap<[u8; 16], ([u8; 32], Instant)>,
37 }
```



如果没有退役表





auth.rs: 把会话放进表里



auth.rs

crates/lanshare/src/server/

第 54-73 行 • 共 73 行

```
54         if self.retired.contains(&cid) || self.pending.contains_key(&cid) {
55             return Admit::Refused;
56         }
57         if self.live.len() >= max
58             && let Some(oldest) = self.live.iter().min_by_key(|(_, s)| s.last_used).map(|(k, _)| *k)
59         {
60             self.live.remove(&oldest);
61             self.retired.insert(oldest);
62         }
63         self.live.insert(
64             cid,
65             Session { keys: ChannelKeys::derive(secret), window: ReplayWindow::new(), last_used: Instant::now() },
66         );
67         Admit::Created
68     }
69
70     pub(crate) fn clear_pending(&mut self) {
71         self.pending.clear();
72     }
73 }
```



限速：v1 数连续失败，v2 记时间戳

Python v1 • auth.py

```
1 count = self._failures.get(ip, 0) + 1
2 if count >= MAX_FAILURES:
3     self._failures.pop(ip, None)
4     self._locked_until[ip] = now + LOCK_SECONDS
5 else:
6     self._failures[ip] = count
```

Rust v2 • auth.rs

```
1 fn recent(q: &mut VecDeque<Instant>, now: Instant) -> usize {
2     while q.front().is_some_and(|t| now.duration_since(*t) > WINDOW) {
3         q.pop_front();
4     }
5     q.len()
6 }
```



审查 I-1：只限速，挡不住慢慢猜

● 全局每分钟 20 次，一天就是 28800 次

● 一天猜中的概率约 2.8%，30 天约 58%

● 原设计“失败 10 次自动换口令”：没用

捣蛋鬼猜的本来就是随机数，换不换口令，每次都是百万分之一

● 修法：累计 30 次没完成的尝试，就暂停口令登录

只能在老电脑的主界面上重新开启，同时换新口令；每开启一次，被猜中的概率不超过百万分之三十



auth.rs: Guard, 管限速和暂停

auth.rs crates/lanshare/src/server/

第 81-100 行 • 共 100 行

```
81 pub(crate) struct Guard {
82     hello_failures: HashMap<IpAddr, VecDeque<Instant>>,
83     pake_attempts: HashMap<IpAddr, VecDeque<Instant>>,
84     pake_global: VecDeque<Instant>,
85     outstanding: usize,
86     paused: bool,
87 }
88
89 impl Guard {
90     pub(crate) fn pin_paused(&self) -> bool {
91         self.paused
92     }
93 }
94
95 fn recent(q: &mut VecDeque<Instant>, now: Instant) -> usize {
96     while q.front().is_some_and(|t| now.duration_since(*t) > WINDOW) {
97         q.pop_front();
98     }
99     q.len()
100 }
```



auth.rs: 三个小工具

auth.rs crates/lanshare/src/server/

第 94-113 行 • 共 113 行

```
94
95 fn recent(q: &mut VecDeque<Instant>, now: Instant) -> usize {
96     while q.front().is_some_and(|t| now.duration_since(*t) > WINDOW) {
97         q.pop_front();
98     }
99     q.len()
100 }
101
102 fn parse_cid(s: &str) -> Option<[u8; 16]> {
103     hex::decode(s).ok()?.try_into().ok()
104 }
105
106 /// 请求体解析成 JSON; 格式不对、字段缺失一律当作 400, 不暴露解析细节。
107 fn parse<T: DeserializeOwned>(body: &Bytes) -> Option<T> {
108     serde_json::from_slice(body).ok()
109 }
110
111 fn error(status: StatusCode, code: &str) -> Response {
112     (status, Json(json!({ "ok": false, "error": code }))).into_response()
113 }
```



握手正文的三条规矩

● 只收 application/json（审查 M-2）

浏览器跨站发 JSON 要先“预检”，服务端不回应预检；text/plain 不用预检，任何网页都能发

● 最多 4 KiB

这几个接口的 JSON 只有几百字节

● 10 秒内收完（第三轮复验 R3-1）

审查实测：一个连接每隔 25 秒发一个字节，100 秒后还挂着



auth.rs: 读握手的小正文

auth.rs crates/lanshare/src/server/

第 119-138 行 • 共 138 行

```
119
120 /// 读握手接口的小正文：限量、限时（第三轮复验 R3-1：不限时的话，一个连接每隔一会儿发一个字节能挂几十个小时）。
121 async fn read_small_body(st: &super::AppState, request: Request) -> Result<(HeaderMap, Bytes), Response> {
122     let (parts, body) = request.into_parts();
123     if !is_json(&parts.headers) {
124         return Err(error(StatusCode::UNSUPPORTED_MEDIA_TYPE, "json_only"));
125     }
126     match tokio::time::timeout(st.handshake_timeout, axum::body::to_bytes(body, HANDSHAKE_MAX)).await {
127         Ok(Ok(bytes)) => Ok((parts.headers, bytes)),
128         Ok(Err(_)) => Err(error(StatusCode::PAYLOAD_TOO_LARGE, "too_large")),
129         Err(_) => Err(error(StatusCode::REQUEST_TIMEOUT, "timeout")),
130     }
131 }
132
133 fn is_json(headers: &HeaderMap) -> bool {
134     headers
135         .get(header::CONTENT_TYPE)
136         .and_then(|v| v.to_str().ok())
137         .is_some_and(|v| v.split(';').next().unwrap_or("").trim().eq_ignore_ascii_case("application/json"))
138 }
```



auth.rs: 扫码握手 hello

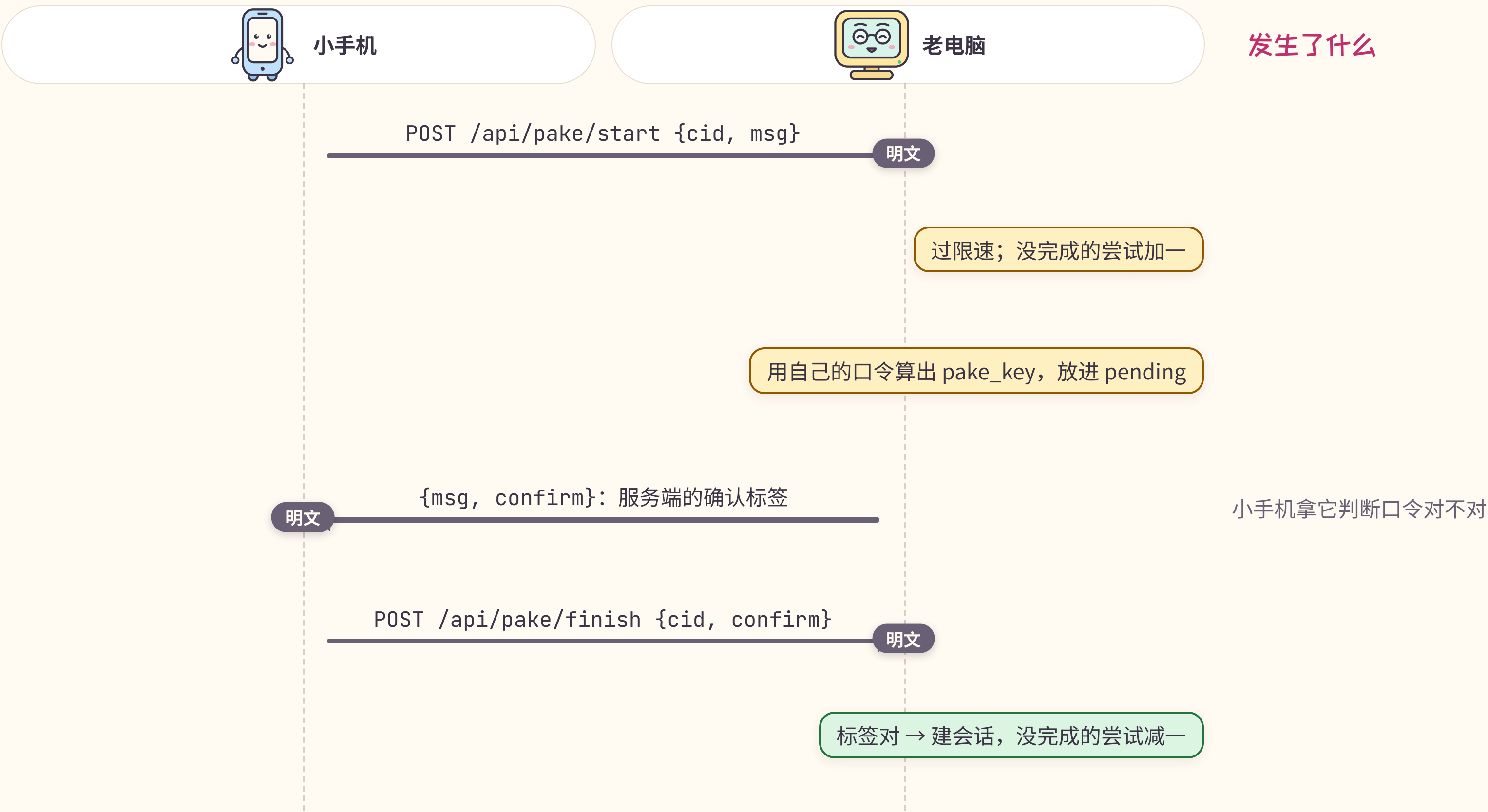
auth.rs crates/lanshare/src/server/

第 160–179 行 • 共 179 行

```
160     }
161 }
162 let Some(body) = parse::<HelloBody>(&body) else { return error(StatusCode::BAD_REQUEST, "bad_request") };
163 let (Some(cid), Ok(tag)) = (parse_cid(&body.cid), hex::decode(&body.tag)) else {
164     return error(StatusCode::BAD_REQUEST, "bad_request");
165 };
166 if !proto::verify_hello(&st.key(), &cid, &tag) {
167     st.guard.lock().expect("锁不会中毒").hello_failures.entry(ip).or_default().push_back(now);
168     st.log(&format!("扫码配对失败: 来自 {ip} 的握手标签不对"));
169     return error(StatusCode::FORBIDDEN, "bad_tag");
170 }
171 let secret = proto::session_secret_from_key(&st.key(), &cid);
172 let admitted = st.sessions.lock().expect("锁不会中毒").admit(cid, &secret, st.max_sessions);
173 match admitted {
174     Admit::Created => st.log(&format!("新设备已配对 (扫码): {ip}")),
175     Admit::AlreadyLive => {}
176     Admit::Refused => return error(StatusCode::CONFLICT, "cid_used"),
177 }
178 Json(json!({ "ok": true, "server_id": st.server_id() })).into_response()
179 }
```



口令配对：老电脑这一侧





复验 N-4：两个漏洞

- 原来先做 SPAKE2 计算，再给“没完成的尝试”加一

几十个请求同时到，都通过了“没暂停”的检查；上限 30，实际到了约 49

- 修法：暂停检查、限速、计数，在同一次加锁里做完，先占位再计算

- 完成配对的接口原来不查暂停

修法：finish 也查，两道防线



auth.rs: 口令配对第一步

auth.rs crates/lanshare/src/server/

第 237-256 行 • 共 256 行

```
237         if sessions.known(&cid) {
238             return error(StatusCode::CONFLICT, "cid_used");
239         }
240         if sessions.pending.len() >= MAX_PENDING_PAKE {
241             return error(StatusCode::TOO_MANY_REQUESTS, "too_many_attempts");
242         }
243         sessions.pending.insert(cid, (pake_key, now));
244     }
245     if pause_now {
246         st.sessions.lock().expect("锁不会中毒").clear_pending();
247         st.log("口令尝试失败次数过多，已暂停口令登录（在电脑上的主界面里可以重新开启）");
248     }
249     let b64 = base64::engine::general_purpose::STANDARD;
250     Json(json!({
251         "ok": true,
252         "msg": b64.encode(&my_msg),
253         "confirm": hex::encode(proto::confirm_tag(&pake_key, "server", &cid)),
254     }))
255     .into_response()
256 }
```



auth.rs: 口令配对第二步

auth.rs crates/lanshare/src/server/

第 282-301 行 • 共 301 行

```
282     };
283     if at.elapsed() >= PAKE_TTL || !proto::verify_confirm(&pake_key, "client", &cid, &tag) {
284         sessions.retired.insert(cid);
285         drop(sessions);
286         st.log(&format!("口令配对失败: 来自 {}", peer.ip()));
287         return error(StatusCode::FORBIDDEN, "bad_confirm");
288     }
289     let secret = proto::session_secret_from_pake(&pake_key, &cid);
290     let admitted = sessions.admit(cid, &secret, st.max_sessions);
291     drop(sessions);
292     if !matches!(admitted, Admit::Created) {
293         return error(StatusCode::CONFLICT, "cid_used");
294     }
295     {
296         let mut guard = st.guard.lock().expect("锁不会中毒");
297         guard.outstanding = guard.outstanding.saturating_sub(1);
298     }
299     st.log(&format!("新设备已配对 (口令): {}", peer.ip()));
300     Json(json!({ "ok": true, "server_id": st.server_id() })).into_response()
301 }
```



mod.rs: 接上 auth

● ● ●

mod.rs

crates/lanshare/src/server/

第 79-98 行 • 共 327 行

```
79         handshake_timeout: Duration::from_secs(10),
80     }
81 }
82 }
83
84 /// 一次运行中所有请求共享的状态。
85 pub struct AppState {
86     key: [u8; 32],
87     server_id: String,
88     pin: Mutex<String>,
89     port: u16,
90     lan_ips: Vec<Ipv4Addr>,
91     pub(crate) storage: Storage,
92     pub(crate) idle_timeout: Duration,
93     pub(crate) max_connections: usize,
94     pub(crate) max_local_connections: usize,
95     pub(crate) max_connections_per_ip: usize,
96     pub(crate) header_timeout: Duration,
97 }
98
```



mod.rs: AppState 带上会话表和 Guard

● ● ●

mod.rs

crates/lanshare/src/server/

第 149–168 行 • 共 343 行

```
149         handshake_timeout: config.handshake_timeout,
150     }
151 }
152
153 pub fn key(&self) -> [u8; 32] {
154     self.key
155 }
156
157 pub fn server_id(&self) -> &str {
158     &self.server_id
159 }
160
161 pub fn pin(&self) -> String {
162     self.pin.lock().expect("口令锁不会中毒").clone()
163 }
164
165 pub fn port(&self) -> u16 {
166     self.port
167 }
168
```



mod.rs: 两个方法，三个接口

● ● ●

mod.rs

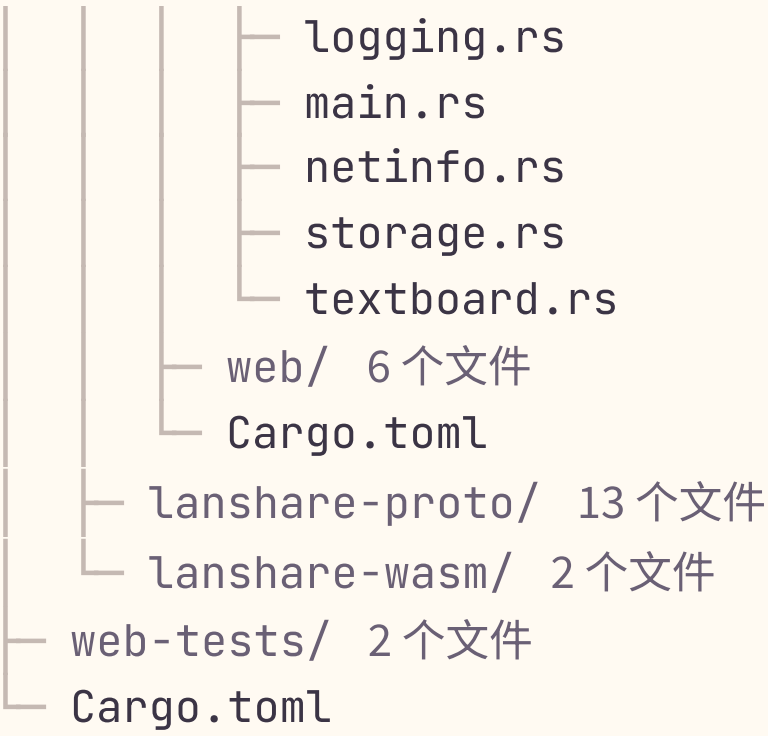
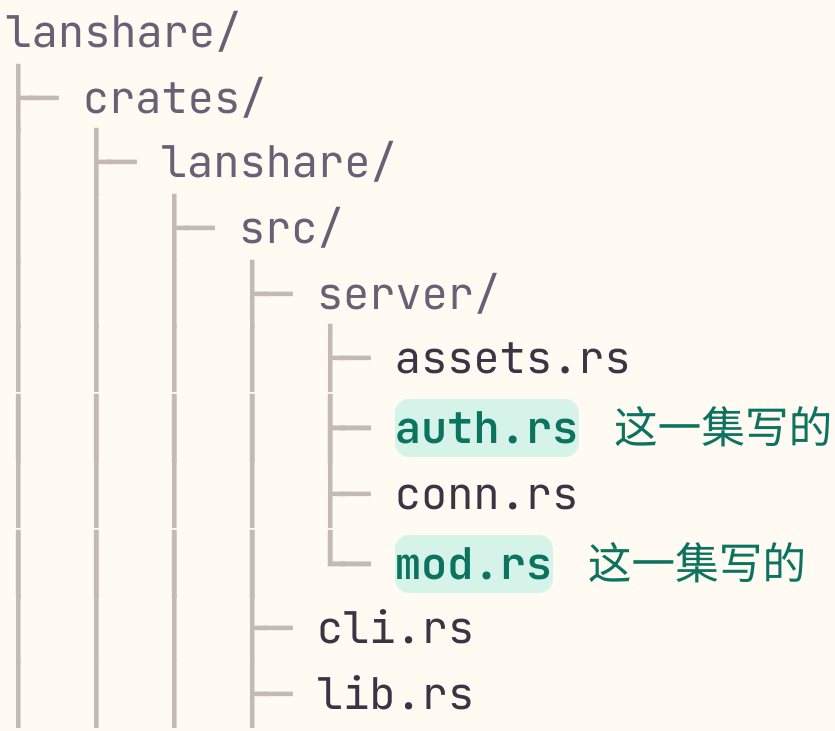
crates/lanshare/src/server/

第 276-295 行 • 共 355 行

```
276     .route("/api/hello", post(auth::hello))
277     .route("/api/pake/start", post(auth::pake_start))
278     .route("/api/pake/finish", post(auth::pake_finish))
279     .layer(middleware::from_fn(check_origin))
280     .layer(middleware::from_fn(security_headers))
281     .layer(middleware::from_fn(check_host))
282     .with_state(state)
283 }
284
285 /// 在已绑定的监听器上提供服务，直到出错。
286 pub async fn serve(listener: tokio::net::TcpListener, state: Arc<AppState>) -> std::io::Result<()> {
287     serve_with_shutdown(listener, state, std::future::pending()).await
288 }
289
290 /// 同上，`shutdown` 完成时停止接受新连接，等进行中的请求结束（最多 3 秒；托盘“退出”时用）。
291 ///
292 /// 没用 `axum::serve`：它不给 hyper 配计时器，读请求头没有时限（独立安全审查 I-3、复验 N-1）。
293 /// 这里自己接收连接：先过准入（每 IP 上限、本机单独一池），再交给 hyper，配上计时器和请求头时限。
294 pub async fn serve_with_shutdown(
295     listener: tokio::net::TcpListener,
```



配对写好了





编译、跑测试

```

Windows PowerShell

PS D:\code\lanshare> cargo build
   Compiling lanshare-proto v2.0.0 (D:\code\lanshare\crates\lanshare-proto)
   Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
   Compiling lanshare-wasm v2.0.0 (D:\code\lanshare\crates\lanshare-wasm)
warning: fields `keys` and `window` are never read
--> crates\lanshare\src\server\auth.rs:26:16
|
25 | pub(crate) struct Session {
|           ^----- fields in this struct
26 |     pub(crate) keys: ChannelKeys,
|                   ^^^^
27 |     pub(crate) window: ReplayWindow,
|                   ^^^^^^^
|
= note: `#[warn(dead_code)]` (part of `#[warn(unused)]`) on by default

warning: `lanshare` (lib) generated 1 warning
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 5.73s

PS D:\code\lanshare> cargo test -p lanshare --lib
   Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)

```



敲一敲握手接口

```
Windows PowerShell

PS D:\code\lanshare> cargo run -p lanshare
   Compiling lanshare v2.0.0 (D:\code\lanshare\crates\lanshare)
warning: fields `keys` and `window` are never read
..... (省略 12 行, 同一条警告)
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.98s
   Running `target\debug\LanShare.exe`
LanShare 骨架跑起来了, 协议版本: lanshare/2
主界面地址: http://127.0.0.1:8765/#yue-NAQA2yIkS-e_IDgCQJJLN70NizuSly6_CoDmNIU

PS D:\code\lanshare> curl.exe -s -w " %{http_code}" -H "Content-Type: text/plain" -d hi http://127.0.0.1:8765/api/hello
{"error":"json_only","ok":false} 415

PS D:\code\lanshare>
curl.exe -s -w " %{http_code}" -H "Content-Type: application/json" -d "{}" http://127.0.0.1:8765/api/hello
{"error":"bad_request","ok":false} 400

PS D:\code\lanshare> Set-Content hello.json '{"cid":"00112233445566778899aabbccddeeff","tag":"00"}'

PS D:\code\lanshare>
1..6 | ForEach-Object { curl.exe -s -w " %{http_code}" -H "Content-Type: application/json" --data-binary "@hello.json"
http://127.0.0.1:8765/api/hello }
{"error":"bad_tag","ok":false} 403
{"error":"bad tag","ok":false} 403
```



选择题

同一个 IP 连续 5 次 hello 标签都不对。第 6 次带着正确的标签来，结果是什么？

A 200，标签对了就放行

B 429，锁定期间正确的也不行

C 403，标签不对



选择题

同一个 IP 连续 5 次 hello 标签都不对。第 6 次带着正确的标签来，结果是什么？

A 200，标签对了就放行

限速检查在验证标签之前，锁定期间根本不看标签。

C 403，标签不对

标签是对的，但请求没有走到验证这一步。

B 429，锁定期间正确的也不行

hello 先查这个 IP 最近 60 秒的失败次数，满 5 次直接 429，还没走到验证标签那一步。



选择题

pake_start 为什么要先把“没完成的尝试”加一，再去做 SPAKE2 计算？

- A 为了让计算更快
- B 让并发的请求不可能一起越过暂停上限
- C 为了少写一次日志



选择题

pake_start 为什么要先把“没完成的尝试”加一，再去做 SPAKE2 计算？

A 为了让计算更快

先后顺序不影响计算速度。

C 为了少写一次日志

和日志无关。

B 让并发的请求不可能一起越过暂停上限

检查暂停和计数在同一次加锁里完成，并发的请求只能一个一个通过。原来先算后计数，几十个请求同时通过“没暂停”的检查，上限 30 实际到了约 49。



选择题

会话表满了，最久没用的会话被淘汰。为什么它的 cid 还要放进 retired，而不是删掉就算？

- A 否则重放旧 hello 能用同一个 cid 重建会话，计数器窗口从零开始，旧请求又能重放
- B 为了统计一共连过多少设备
- C 为了让小手机下次连得更快



选择题

会话表满了，最久没用的会话被淘汰。为什么它的 cid 还要放进 retired，而不是删掉就算？

- A** 否则重放旧 hello 能用同一个 cid 重建会话，计数器窗口从零开始，旧请求又能重放

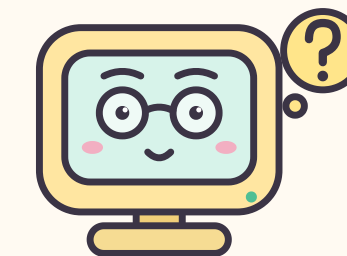
新会话的防重放窗口什么都没见过。cid 进了退役表，本次运行内永不再收，重放旧 hello 只会拿到 409。

- B** 为了统计一共连过多少设备

退役表是安全措施，不是统计用的。

- C** 为了让小手机下次连得更快

恰恰相反，这个 cid 再也不能用了，小手机会换一个新 cid 重新握手。



捣蛋鬼在局域网里不停地发 pake_start，故意让口令登录暂停。这算不算漏洞？用户会受到什么影响？



想一想

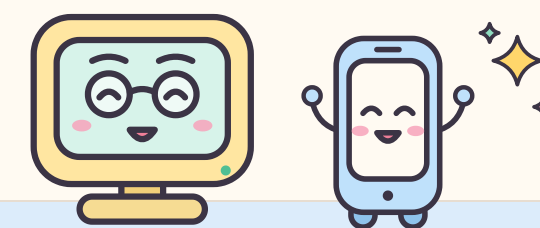
捣蛋鬼在局域网里不停地发 pake_start，故意让口令登录暂停。这算不算漏洞？用户会受到什么影响？

这是 SECURITY.md 里如实写着的“局域网内的拒绝服务”：口令登录会被暂停，需要有人在老电脑的主界面上重新开启。但它不会泄密：每开启一次，捣蛋鬼最多只能验证 30 个口令。扫码登录也完全不受影响：扫码走的是 hello，它根本不看暂停标志。它也不需要看：扫码要出示的是用 32 字节密钥算出的标签，猜不出来。这是一个有意的取舍：宁可让口令登录暂时用不了，也不能让它被慢慢猜中。



这一集写了什么

- ✓ 会话表三张：live、pending、retired；被淘汰的 cid 永不再收，重放旧握手复活不了会话
- ✓ hello：先限速，再验标签，再建会话；重放的 hello 不重置任何状态
- ✓ 口令配对两步：start 先占位再计算，finish 再查一次暂停（复验 N-4）
- ✓ 限速用 VecDeque 滑动窗口；累计 30 次没完成就暂停，只管速度不够，还要管总量（审查 I-1）
- ✓ 握手正文只收 JSON、最多 4 KiB、10 秒收完（审查 M-2、第三轮复验 R3-1）



下一集

全课完结，感谢收看